

KỊCH BẢN QUAY VIDEO DEMO NỘP BÀI (VIDEO DEMO PRESENTATION SCRIPT)

Mã bài tập: HW05 – Performance Testing (Exercise ID: HW05-AI)

Sinh viên thực hiện: BaoBeiii – MSSV: 23127327

Email: 23127327@student.hcmus.edu.vn

Thời lượng video khuyến nghị: 3 đến 5 phút

Định dạng quay: Màn hình máy tính + Giọng thuyết minh (Screen Recording with Audio Commentary)

[!IMPORTANT]

LIÊN KẾT VIDEO DEMO CHÍNH THỨC (OFFICIAL DEMO LINKS)

-  Video Demo Task 1 (Đo tải k6, Bẫy 7 bug SUT, Phản biện AI & HTML Dashboard): <https://youtu.be/leNk4TxJ1D4>
-  Video Demo Agent Skill (Đóng gói Agent Skill Antigravity, CLI Metrics Gate & CI/CD): <https://youtu.be/vSyRLXkW-7k>

PHÂN BỐ THỜI LƯỢNG TỔNG QUAN

[0:00 - 0:30] Phần 1: Giới thiệu bản thân & Tổng quan cấu trúc đề án
[0:30 - 1:30] Phần 2: Demo thực thi k6 đo tải & Trình chiếu HTML Dashboard
[1:30 - 2:30] Phần 3: Soi mã nguồn SUT & Bảng chứng bắt 7 lỗi hệ thống
[2:30 - 3:30] Phần 4: Phản biện lỗi suy diễn của AI & 4 giải pháp tối ưu kiến trúc
[3:30 - 4:30] Phần 5: Demo Tự động hóa CI/CD GitHub Actions & Agent Skill
[4:30 - 5:00] Phần 6: Tổng kết, đối chiếu Rubric 100/100 & Lời kết

KỊCH BẢN CHI TIẾT TỪNG PHẦN ĐOẠN (SCENE-BY-SCENE)

PHẦN 1: GIỚI THIỆU BẢN THÂN & TỔNG QUAN CẤU TRÚC ĐỀ ÁN (0:00 – 0:30)

- Hành động trên màn hình:

- i. Mở file [README.md](#) hiển thị rõ Họ tên, MSSV: **23127327**.
- ii. Mở cây thư mục VS Code hiển thị rõ ràng: `data/` , `scripts/` , `results/` , `evidence/` , `report.md` , `.agents/` , `.github/` .

- **Lời thoại (Voice-over):**

"Kính chào thầy cô và các bạn! Em là sinh viên thực hiện đồ án HW05 Kiểm thử hiệu năng ứng dụng Web với sự hỗ trợ của AI, MSSV: **23127327**.
Dự án được xây dựng theo chiến lược AI-First kết hợp Phản biện Con người sâu sắc. Toàn bộ kịch bản đo tải, bộ dữ liệu tham số hóa, báo cáo kết quả và pipeline CI/CD đều được em tổ chức bài bản, cô lập hoàn toàn với repo nguồn SUT và được lưu trữ sạch sẽ trong repository này."

PHẦN 2: DEMO THỰC THI ĐO TẢI & TRÌNH CHIẾU HTML DASHBOARD (0:30 – 1:30)

- **Hành động trên màn hình:**

- i. Mở Terminal PowerShell, hiển thị backend Node.js đang chạy trên cổng 3000.
- ii. Trình chiếu lệnh k6 chạy kiểm thử.
- iii. Mở trình duyệt hiển thị file [results/load/summary.html](#) và [results/stress/summary.html](#).
- iv. rê chuột chỉ vào biểu đồ Response Time phân vị \$p95\$ và các bảng thống kê.

- **Lời thoại (Voice-over):**

"Em đã thực thi đo tải thực tế cho cả 4 kịch bản: Load Test 50 VUs, Stress Test 200 VUs, Spike Test 150 VUs và Endurance Test 15 phút.
Nhờ tích hợp k6-reporter offline, hệ thống tự động xuất ra Dashboard HTML trực quan tương tác cao cấp này.
Tại bài Load Test, hệ thống đạt thông lượng 10.6 req/s với độ trễ p95 chỉ 1.74ms, đạt chuẩn SLA đề bài.
Tại bài Stress Test, em đã xác định chính xác Điểm gãy (Breaking Point) nằm ở dải 120 đến 150 VUs khi SQLite bị nghẽn khóa ghi hàng loạt."

PHẦN 3: SOI MÃ NGUỒN SUT & BẢNG CHỨNG BẮT 7 LỖI HỆ THỐNG (1:30 – 2:30)

- **Hành động trên màn hình:**

- i. Mở file `eshop-sut/backend/server.js` , cuộn đến dòng 398 (BUG-01) và dòng 162 (BUG-03).
- ii. Mở file [bug_reports.md](#).

iii. Mở file [evidence/load_test_metrics.txt](#) chỉ vào số lượng lỗi k6 đã bắt được.

- **Lời thoại (Voice-over):**

"Bằng cách soi trực tiếp mã nguồn backend, em phát hiện SUT có tới 7 lỗi hệ thống nghiêm trọng mà AI ban đầu đã bỏ sót.

Điển hình như BUG-01 tại dòng 398: Công thức giảm giá phần trăm bị nhân ngược ($1 - \text{discount_value}$), khiến số tiền giảm bị âm và đội giá đơn hàng lên gấp 10 lần.

Hay BUG-03 tại dòng 162: Backend cố tình ép kiểu giá thành chuỗi String ở các ID sản phẩm chẵn.

Em đã viết các Deep Assertions trên k6 và bắt trọn hơn 4.000 lần xuất hiện của các lỗi này trong quá trình đo tải thực nghiệm."

PHẦN 4: PHẢN BIỆN LỖI SUY DIỄN AI & 4 GIẢI PHÁP TỐI ƯU KIẾN TRÚC (2:30 – 3:30)

- **Hành động trên màn hình:**

i. Mở file [report.md](#) (Mục 6).

ii. Cuộn qua phần đối chiếu AI raw response và Human Critique.

iii. Dừng lại ở 4 giải pháp tối ưu hóa kiến trúc.

- **Lời thoại (Voice-over):**

"Trong Task 2, khi đưa log cho AI phân tích, AI đã mắc 5 sai lầm kinh điển:

Thứ nhất là ngộ biện giá trị trung bình (The Mean Fallacy), ca ngợi Average Latency 1.1ms mà bỏ qua độ trễ p95 ở bước Checkout chậm gấp 6 lần.

Thứ hai là ngộ nhận tỷ lệ lỗi 42.8% là do server sập, trong khi thực tế đó là các ca test âm tính có chủ đích.

Thứ ba là chẩn đoán sai rò rỉ RAM; chuỗi 576 mẫu telemetry của em chứng minh Garbage Collection hoạt động hoàn hảo và drift chỉ +10.7MB sau 32 phút.

Thay vì các lời khuyên sáo rỗng của AI như 'viết lại bằng Rust hay dùng K8s', em đã đề xuất 4 giải pháp sát sườn: Bật SQLite WAL mode giải quyết nghẽn khóa ghi, Đánh B-Tree Indexing giảm Full Table Scan, Sửa công thức coupon kèm Transaction nguyên tử, và Cài đặt Cache In-Memory cho danh mục sản phẩm."

PHẦN 5: DEMO CI/CD GITHUB ACTIONS & AGENT SKILL (3:30 – 4:30)

- **Hành động trên màn hình:**

i. Mở file [.github/workflows/performance-regression.yml](#).

ii. Mở thư mục [.agents/skills/performance-testing/](#).

- iii. Mở Terminal chạy lệnh: `node .agents/skills/performance-testing/scripts/extract_metrics.js results/load/metrics.json`
- iv. Hiển thị bảng Markdown đánh giá SLA Gate in ra terminal với thông báo `GATE APPROVED`.

- **Lời thoại (Voice-over):**

"Đối với Task 3, em đã thiết lập workflow GitHub Actions hoàn chỉnh, tự động clone SUT, thăm dò sức khỏe và bắn tải k6 mỗi khi có Pull Request.

Đồng thời, em đã đóng gói trọn bộ Agent Skill `performance-testing` chuẩn Antigravity.

Như thầy cô đang thấy trên màn hình, khi em thực thi script `extract_metrics.js`, tool sẽ tự động đọc file `metrics.json`, bóc tách trể p95, so sánh với ma trận SLA và in ra bảng đánh giá Markdown chuyên nghiệp này. Nếu có bất kỳ endpoint nào bị thoái hóa độ trể, script sẽ trả về mã lỗi 1 để tự động chặn merge PR."

PHẦN 6: TỔNG KẾT & LỜI KẾT (4:30 – 5:00)

- **Hành động trên màn hình:**

- i. Mở file [README.md](#) (Mục 9) hiển thị bảng tự chấm điểm 100/100.
- ii. Mở file [AI_Audit_Report.md](#) lướt qua bảng tổng kết 6 phiên làm việc với 4-5 điểm sửa đổi cụ thể cho từng phần.
- iii. Mở `git log --oneline` hiển thị chuỗi commit sạch sẽ và đều đặn.

- **Lời thoại (Voice-over):**

"Tổng kết lại, đồ án đã hoàn thành trọn vẹn 100% các yêu cầu từ Task 1, Task 2 đến Task 3. Báo cáo kiểm toán AI Audit Report ghi nhận mình bạch 5 phiên làm việc với từ 4 đến 5 sửa đổi mang tính quyết định của con người cho mỗi task.

Em tự tin tự đánh giá bài làm đạt điểm số tối đa 100/100 theo đúng Rubric của môn học.

Em xin chân thành cảm ơn thầy cô đã theo dõi video báo cáo của em!"